

ASSIGNMENT

Name	SHAIK THOUHID
Register Number	192421280
Course Code	ITA0609

1. K-means clustering produces inconsistent cluster assignments for similar datasets. Analyze the reasons and suggest debugging and optimization strategies. (5 Marks)

Introduction: K-means is a distance-based clustering algorithm. Its output may change even when similar datasets are used because the algorithm is highly dependent on the starting centroids and the structure of the data.

Possible reasons: Poor initialization can place centroids in unsuitable positions and lead to different local solutions. An incorrect value of K may force unrelated points into one cluster or split a natural group. Outliers can pull centroids away from dense regions. Features measured on different scales may dominate the distance calculation. Overlapping or non-spherical clusters can also reduce consistency.

Debugging approaches: Run the algorithm several times with different random seeds and compare the inertia values. Plot the clusters and centroids to identify unstable or overlapping regions. Check the dataset for missing values, duplicate records, extreme outliers, and large differences in feature ranges. Use the elbow method and silhouette score to verify whether the selected number of clusters is suitable.

Optimization strategies: Use K-means++ to choose better initial centroids. Normalize or standardize all numerical features before clustering. Remove or cap serious outliers when justified. Increase the number of initial runs using a higher n_init value and choose the result with the lowest within-cluster sum of squares. Select K using elbow and silhouette analysis. If clusters are irregular or strongly affected by noise, consider DBSCAN or hierarchical clustering.

Conclusion: K-means becomes more stable when initialization, scaling, outliers, and the number of clusters are carefully checked. K-means++, normalization, repeated runs, and silhouette analysis are effective ways to improve the final clustering result.

2. An SVM model underperforms on a nonlinear dataset. Identify the issues and suggest debugging and optimization methods. (5 Marks)

Introduction: A Support Vector Machine finds a decision boundary that maximizes the margin between classes. A basic linear SVM may perform poorly when the relationship between features and classes is nonlinear.

Possible issues: The main problem may be an unsuitable kernel, such as using a linear kernel for curved class boundaries. The regularization parameter C may be too high or too low. For an RBF kernel, an unsuitable γ value can cause underfitting or overfitting. Unscaled features can distort the distance calculation. Noisy, irrelevant, or highly correlated features may also reduce performance. Class imbalance can make the model favor the majority class.

Debugging techniques: First compare training and validation accuracy. Low scores on both indicate underfitting, while very high training accuracy with poor validation accuracy indicates overfitting. Plot the data in two dimensions when possible to observe whether the boundary is nonlinear. Check feature distributions and scaling. Use a confusion matrix, precision, recall, and F1-score instead of accuracy alone, especially for imbalanced classes.

Optimization methods: Test suitable kernels such as RBF, polynomial, and sigmoid, while keeping the linear kernel as a baseline. Standardize the data using z-score scaling before training. Apply grid search or randomized search with cross-validation to tune C , γ , kernel type, and polynomial degree. Use `class_weight="balanced"` or resampling when classes are unequal. Remove irrelevant features or apply dimensionality reduction if the feature space is very large.

Conclusion: For nonlinear data, performance usually improves through proper kernel selection, feature scaling, and systematic hyperparameter tuning. Cross-validation helps select a model that generalizes well instead of only fitting the training set.

3. An RNN-based NLP model suffers from poor long-term dependency learning. Examine the causes and propose debugging steps and solutions. (5 Marks)

Introduction: Recurrent Neural Networks process text one token at a time and pass hidden information from one step to the next. Standard RNNs often struggle to remember information from words that appeared much earlier in a long sequence.

Main causes: During backpropagation through many time steps, gradients can repeatedly become very small, producing the vanishing-gradient problem. Important early information is then not learned. Gradients may also become extremely large, causing unstable training. A simple RNN has limited memory capacity, and long sequences increase the difficulty of preserving useful context. Poor tokenization, insufficient data, long padded sequences, or unsuitable learning rates can make the problem worse.

Debugging steps: Monitor training and validation loss across epochs and check whether learning stops early or becomes unstable. Track gradient norms to detect vanishing or exploding gradients. Test the model on short and long sentences separately. Inspect the maximum sequence length, padding method, vocabulary size, and out-of-vocabulary rate. Compare the standard RNN with an LSTM or GRU baseline using the same dataset.

Proposed solutions: Replace the basic RNN with LSTM or GRU units because their gates control what information should be stored, forgotten, or passed forward. Use bidirectional LSTM or GRU when both previous and later context are useful. Add an attention mechanism so the model can directly focus on important words instead of depending only on the final hidden state. Apply gradient clipping to control exploding gradients, use proper weight initialization, tune the learning rate, and use optimizers such as Adam. Bucketing sequences by length and using appropriate dropout can also improve training.

Conclusion: Poor long-term learning in RNNs mainly comes from gradient problems and limited memory. LSTM or GRU units, attention, gradient clipping, careful sequence preparation, and better training settings provide more reliable learning for long text sequences.